LOADPILOT — PLATEFORME DE TEST DE CHARGE

Rapport technique
Module Scénario

Gestion des scénarios, des étapes HTTP et de leur exécution en charge : architecture, moteur, base de données, sécurité, suppression/archivage et tests

BACKEND
Java 21 · Spring Boot 3.3.4

FRONTEND
React 18.3 · TypeScript 5.9 · Vite 5.4

DONNÉES
PostgreSQL 17.5 · Liquibase 4.27

SÉCURITÉ
Keycloak 26.7 · OAuth2 / JWT

MOTEUR
java.net.http · threads virtuels

TESTS
605 backend · 51 frontend · 0 échec

Sommaire

1. Vue d'ensemble
2. Backend
3. Moteur d'exécution
4. Frontend
5. Base de données
6. Sécurité
7. Suppression et archivage
8. Flux complets
9. Exemple concret sur banking-test-api
10. Tests
11. Limites et améliorations possibles

1. Vue d'ensemble

Module Scénario howa l'qalb dyal LoadPilot : fih kat3erref **chno ghadi ttesti** (scénario = suite d'étapes HTTP 3la wa7ed l'application), **b ch7al men charge** (utilisateurs virtuels, ramp-up, durée wla itérations, débit cible), w kat**lanci l'exécution** li katsift vraies requêtes HTTP w katsjel kol résultat bach t3ti rapport statistique.



Fonctionnalités présentes dans le code

Fonctionnalité	Ce qui existe réellement
Scénario	CRUD, paramètres de charge (VU, ramp-up, durée/itérations, think time, débit cible `targetRps`), données CSV par VU, archivage/restauration, suppression définitive en cascade, impact avant suppression.
Étapes HTTP	GET/POST/PUT/PATCH/DELETE, URL relative ou absolue, en-têtes JSON, corps, code attendu, assertion « contient », timeout et think time par étape, redirections, capture de variable (JSONPath simple), ordre renuméroté après suppression.
Exécution	Asynchrone (202), suivi du statut, annulation réelle, relance, limites de capacité, récupération au redémarrage, planification (unique ou CRON).
Authentification	Celle de l'application (Bearer, clé API, Basic, connexion auto) ajoutée à chaque requête ; jeton disponible via `${authToken}`.
Résultats	Détail par requête, rapport (min/max/moyenne, P50/P75/P90/P95/P99, écart-type, débit, taux d'erreur, par étape), export CSV, historique paginé, métriques.
Interface	Page Scénarios (liste, détail, étapes, édition, exécution, suppression), assistant « Nouveau scénario » en 4 étapes avec test réel, pages Exécutions, Détail, Rapport, Historique, Planification.

2. Backend

Technologies (versions résolues par Maven)

Technologie	Version	Rôle dans le module
Java	21.0.6	Records (DTO, `StepExecutionSpec`, `LoadTestSpec`), **threads virtuels** pour les utilisateurs virtuels.
Spring Boot	3.3.4	Démarrage, configuration (`application.yml` : timeouts, limites), injection.
Spring Web MVC	6.1.13	Contrôleurs REST `/api/scenarios`, `/api/steps`, `/api/executions`, `/api/scheduled-executions`, `/api/metrics`.
Spring Data JPA / Hibernate	3.3.4 / 6.5.3.Final	Persistance des entités, requêtes JPQL, `Specification` pour l'historique filtré.
Liquibase	4.27.0	Migrations 003 à 017 du module (voir section 5).
Driver PostgreSQL	42.7.4	Connexion JDBC à PostgreSQL 17.5.
MapStruct	1.6.2	`ScenarioMapper`, `StepMapper`, `ExecutionMapper`, `ExecutionStepResultMapper`, `MetricMapper`.
Lombok	1.18.34	Getters/setters/builders des entités.
Jackson	2.17.2	Lecture des en-têtes JSON des étapes, extraction JSONPath, réponses REST.
Spring Security	6.3.3	JWT Keycloak, `@PreAuthorize` par endpoint.
java.net.http.HttpClient	JDK 21	Envoi des vraies requêtes HTTP (HTTP/1.1, timeouts, redirections).
springdoc-openapi	2.6.0	Documentation Swagger des endpoints.
Tomcat embarqué	10.1.30	Serveur HTTP de l'API.

Architecture en couches

```
ScenarioController · StepController · ExecutionController · ScheduledExecutionController · MetricController
        ↓ DTO (ScenarioRequest, StepRequest, ExecutionRequest…) + Bean Validation
ScenarioServiceImpl · StepServiceImpl · ExecutionServiceImpl · DeletionServiceImpl ·
ScheduledExecutionServiceImpl
        ↓                    ↘ ExecutionTransactionHelper (transactions courtes) → HttpClientExecutionEngine
ScenarioMapper · StepMapper · ExecutionMapper · ExecutionStepResultMapper (MapStruct)
        ↓
ScenarioRepository · StepRepository · ExecutionRepository · ExecutionStepResultRepository · MetricRepository
        ↓
Scenario · Step · Execution · ExecutionStepResult · Metric · ScheduledExecution (entités JPA)
```

Classes principales

Classe	Rôle
`entity/Scenario`, `Step`	Scénario (paramètres de charge, CSV, archivage) et étape HTTP (ordre, options moteur, capture).
`entity/Execution`, `ExecutionStepResult`	Exécution (copie figée des paramètres de charge, statut, compteurs) et résultat par requête (avec copie nom/méthode/URL de l'étape).
`entity/Metric`, `ScheduledExecution`	Métrique par résultat (temps, code, débit, taux d'erreur) ; planification unique ou CRON.

Classe	Rôle
`ScenarioServiceImpl`, `StepServiceImpl`	CRUD ; listes sans archivés ; suppression d'étape (refus si exécution en cours, renumérotation).
`ExecutionServiceImpl` (515 l.)	Lancement asynchrone, annulation, relance, détail, rapport, historique, export CSV.
`ExecutionTransactionHelper`	Transactions courtes : préparation (refus si archivé ou sans étape), QUEUED→RUNNING, finalisation des résultats.
`HttpClientExecutionEngine` (430 l.)	Moteur de charge réel (section 3).
`RunningExecutionRegistry`	Limites de capacité (VU globaux, exécutions simultanées) et annulation.
`ExecutionRecoveryRunner`	Au démarrage, passe en FAILED les exécutions restées QUEUED/RUNNING.
`DeletionServiceImpl`	Impact, archivage, restauration, suppression définitive en cascade (section 7).
`PerformanceStatisticsService`, `PercentileCalculator`	Statistiques du rapport calculées à la demande.
`MetricGenerationService`	Génère une métrique par résultat à la fin de l'exécution.
`ScheduledExecutionPoller` / `Trigger`	Déclenchement des planifications (intervalle 5 s par défaut).
`CsvDataSource`, `VariableResolver`, `JsonPathExtractor`, `PacingGate`, `UrlResolver`	Données CSV par VU, substitution `${nom}`, capture JSON, débit cible partagé, résolution des URL.

Endpoints REST du module

« Authentifié » = tout utilisateur connecté (VIEWER inclus). SA = SUPER_ADMIN, PE = PERFORMANCE_ENGINEER.

Méthode	URL	Rôle	Description
GET	`/api/scenarios[?applicationId&archived]`	Authentifié	Lister (archivés masqués par défaut)
GET	`/api/scenarios/{id}`	Authentifié	Détail d'un scénario
POST	`/api/scenarios`	SA, PE	Créer
PUT	`/api/scenarios/{id}`	SA, PE	Modifier
GET	`/api/scenarios/{id}/deletion-impact`	SA, PE	Impact (étapes, exécutions, planifications, en cours)
POST	`/api/scenarios/{id}/archive` · `/restore`	SA, PE	Archiver / restaurer
DELETE	`/api/scenarios/{id}`	SA	Suppression définitive en cascade
GET	`/api/steps[?scenarioId]` · `/api/steps/{id}`	Authentifié	Lister / détail des étapes
POST · PUT	`/api/steps` · `/api/steps/{id}`	SA, PE	Créer / modifier une étape
DELETE	`/api/steps/{id}`	SA, PE	Supprimer (refus si exécution en cours)
POST	`/api/executions`	SA, PE	Lancer (réponse 202, QUEUED/RUNNING)
GET	`/api/executions` · `/{id}` · `/{id}/status`	Authentifié	Liste, détail avec résultats, statut léger
GET	`/api/executions/history`	Authentifié	Historique paginé, filtré, trié côté serveur
GET	`/api/executions/{id}/report` · `/report/export`	Authentifié	Rapport statistique / export CSV
POST	`/api/executions/{id}/cancel` · `/retry`	SA, PE	Annuler / relancer
GET · POST · PUT · DELETE	`/api/scheduled-executions[/{id}]`	Lecture : authentifié ; écriture : SA, PE	Planifications
PATCH · POST	`/{id}/enable` · `/{id}/disable` · `/{id}/run-now`	SA, PE	Activer, désactiver, déclencher
GET	`/api/metrics` · `/api/metrics/{id}`	Authentifié	Métriques filtrables

Règles de validation (DTO)

Champ	Règle réelle
Scénario : nom / description	Obligatoire, 255 car. max / 1000 car. max.
virtualUsers · rampUpSeconds	DTO : 1 à 10 000 · ≥ 0. Puis le service refuse (400) au-delà de la limite du déploiement, 500 par défaut (`LOADPILOT_MAX_VUS_PER_EXECUTION`).
durationSeconds · iterations · targetRps	Optionnels, ≥ 1 si fournis.
csvData	50 000 car. max.
Étape : nom, URL, ordre	Obligatoires ; code attendu entre 100 et 599 ; assertion 500 car. ; capture : nom 255, chemin 500.

3. Moteur d'exécution

Moteur kaykhdem **dakhel nfs JVM dyal backend** : ma kaynch outil externe (JMeter/Gatling). Kol utilisateur virtuel (VU) kaykhdem f **thread virtuel** dyal Java 21 (`Executors.newVirtualThreadPerTaskExecutor()` f `ExecutorConfig`).

```
POST /api/executions {scenarioId}
  │ ExecutionTransactionHelper.prepareAndStart   (transaction courte)
  │   • refus si scénario/application archivé, ou 0 étape
  │   • Execution QUEUED + copie des paramètres de charge
  │   • copie de l'authentification de l'application
  ▼ réponse 202 immédiate ; la suite est asynchrone
HttpClientExecutionEngine.execute(steps, baseUrl, auth, loadSpec, handle, csvRows)
  │ 1. authentification résolue UNE fois (login réel si « connexion auto »)
  │ 2. VU 1..N démarrés avec un décalage = rampUp / N
  │ 3. chaque VU : itérations jusqu'à durée / nombre d'itérations / annulation
  │      étape → requête HTTP → StepOutcome (code, temps, succès, erreur)
  │      une étape en échec arrête l'itération en cours de ce VU
  ▼
finalizeExecution : ExecutionStepResult (+ copie de l'étape), statut SUCCESS/FAILED/CANCELLED
MetricGenerationService : une Metric par résultat (temps, code, débit, taux d'erreur)
```

Aspect	Comportement réel
Utilisateurs virtuels	N threads virtuels ; décalage de démarrage `rampUpSeconds × 1000 / N` ms ; variables CSV distribuées de façon **cyclique** (une ligne par VU).
Fin du test	Durée (`durationSeconds`), nombre d'itérations, annulation, ou 1 seul passage si aucun des deux.
Débit cible	`PacingGate` partagé par **tous** les VU (pas un par VU).
Succès d'une étape	Code = code attendu, sinon 200–399 ; plus l'assertion « la réponse contient » si configurée.
Timeouts	10 s par défaut (`EXECUTION_TIMEOUT_SECONDS`), surchargeable par étape. Client HTTP/1.1.
Authentification	Bearer / clé API (en-tête ou paramètre d'URL) / Basic / connexion auto ; ajoutée à **chaque** requête. Login refusé → exécution FAILED « Authentification : … », aucune étape jouée.
`${authToken}`	Jeton obtenu par la connexion auto, injecté dans les variables de chaque VU (utilisable dans URL, en-têtes, corps).
Priorité des en-têtes	Un en-tête défini par l'étape (comparaison insensible à la casse) remplace celui de l'application.
Capture de variable	`JsonPathExtractor` : chemins à points (`$.token` , `$.data.id`), pas de tableaux ; variable propre à chaque VU.
Corps de réponse	Lu **seulement** si une assertion ou une capture est configurée (sinon ignoré pour économiser la mémoire).
Données collectées	Par requête : code HTTP, temps de réponse (ms), succès, erreur, horodatage ; puis métriques et statistiques.
Limites	500 VU max par scénario (refus 400 à l'enregistrement) ; au lancement, 200 VU simultanés au total et 10 exécutions simultanées (refus HTTP 429). Valeurs configurables.
Robustesse	Annulation réelle (interruption des threads) ; au redémarrage, les exécutions orphelines passent en FAILED avec une raison.

4. Frontend

Technologie	Version installée	Rôle
React / React DOM	18.3.1	Pages et composants du module.
TypeScript	5.9.3	Types `BackendScenarioResponse`, `BackendStepRequest`, `BackendExecutionReportResponse` … alignés sur les DTO Java.
Vite	5.4.21	Serveur de dev (port 3000) et build.
React Router	6.30.4	Routes `/scenarios`, `/scenarios/new`, `/executions`, `/executions/detail/:id`, `/executions/report/:id`, `/historique`, `/planification`.
Chart.js + react-chartjs-2	4.5.1	Graphiques du rapport (barres, anneau).
Bootstrap / Bootstrap Icons	5.3.8 / 1.13.1	Grille et icônes.
Vitest + Testing Library	2.1.9 / 16.3.3	Tests frontend.

Organisation des fichiers

```
pages/Scenarios.tsx (1483 l.)           liste, détail + étapes, édition, exécution, suppression
pages/scenario-wizard/
   ScenarioWizard.tsx (619 l.)          assistant « Nouveau scénario » (4 étapes)
   StepEditor.tsx · TestRunPanel.tsx · LoadSummaryPanel.tsx · wizardModel.ts · scenarioWizard.css
pages/Executions.tsx · ExecutionDetail.tsx · ExecutionReport.tsx · Historique.tsx · ScheduledExecutions.tsx
components/Pagination.tsx               pagination commune (taille de page optionnelle)
components/deletion/DeletionDialog.tsx  modale d'impact (archiver / supprimer définitivement)
services/api/scenariosBackend.ts · stepsBackend.ts · executionsBackend.ts · scheduledExecutionsBackend.ts
types/backendContracts.ts               contrats TypeScript = DTO Java
```

Pages et composants

Écran	Ce qui existe réellement
Liste des scénarios	Recherche (nom, application), filtre de statut, case « Afficher les archivés », pagination 5/10/25/50 avec page et taille dans l'URL, aperçu des étapes et dernière exécution par ligne.
Création	Assistant en 4 étapes : Informations (select d'application) → Étapes (éditeur à onglets En-têtes, Corps, Validation, Variables ; glisser-déposer) → Tester (exécution réelle 1 VU × 1 passage, résultat par étape) → Charge & résumé (presets Smoke/Charge/Stress/Personnalisé).
Modification	Modale d'édition du scénario (paramètres de charge, CSV, débit cible) ; modale d'étape (création, modification, suppression) dans le détail.
Lancement	Bouton Exécuter par ligne, « Enregistrer et lancer » dans l'assistant ; pages Exécutions et Détail avec suivi (polling), Annuler, Relancer.
Résultats	Détail par requête ; Rapport (statistiques, percentiles, par étape, erreurs, graphiques, export CSV) ; Historique paginé côté serveur.
Suppression	`DeletionDialog` : impact, Archiver (par défaut), Supprimer définitivement (SA, saisie du nom) ; Restaurer dans la vue archivée.
Tri	Liste des scénarios triée côté backend du plus récent au plus ancien ; **pas de sélecteur de tri** sur cette page.
Sélection multiple	Cases à cocher présentes, mais **aucune action groupée** n'y est branchée (partiel).

5. Base de données

Schéma relevé sur la base PostgreSQL 17.5 réelle (information_schema).

Table `scenario`

Colonne	Type	Remarque
id · application_id · created_by	UUID	Clé primaire ; FK application ; FK app_user. NOT NULL.
name · description · status	VARCHAR(255) · (1000) · (20)	status = ACTIVE/INACTIVE.
virtual_users · ramp_up_seconds · think_time_ms	INTEGER	NOT NULL, défauts 1 · 0 · 0.
duration_seconds · iterations · target_rps	INTEGER	Optionnels.
csv_data	VARCHAR(50000)	Jeu de données CSV.
archived · archived_at	BOOLEAN · TIMESTAMPTZ	Archivage logique (défaut false).
created_at · updated_at	TIMESTAMPTZ	Traçabilité.

Table `step`

Colonne	Type	Remarque
id · scenario_id	UUID	FK scenario, NOT NULL.
name · method · url	VARCHAR(255) · (10) · (2048)	NOT NULL.
headers · body	VARCHAR(4000)	En-têtes au format JSON ; corps libre.
step_order · expected_status	INTEGER	Ordre NOT NULL (non unique) ; code attendu optionnel.
think_time_ms · timeout_seconds · follow_redirects	INTEGER · INTEGER · BOOLEAN	Options par étape (optionnelles).
assertion_body_contains	VARCHAR(500)	Assertion de contenu.
capture_variable_name · capture_json_path	VARCHAR(255) · (500)	Capture de variable.
status · created_at · updated_at	VARCHAR(20) · TIMESTAMPTZ	status = ACTIVE/INACTIVE.

Tables `execution` et `execution_step_result`

Colonne	Type	Remarque
execution.scenario_id · triggered_by_app_user_id	UUID	FK scenario (NOT NULL) ; FK app_user (optionnelle).
execution.status · error_message	VARCHAR(20) · (2000)	QUEUED, RUNNING, SUCCESS, FAILED, CANCELLED.
execution.started_at · finished_at · duration_ms	TIMESTAMPTZ · BIGINT	Horodatage et durée.

Colonne	Type	Remarque
execution.total/successful/failed_steps	INTEGER	Compteurs de requêtes.
execution.virtual_users · ramp_up_seconds · duration_seconds · iterations	INTEGER	Copie figée des paramètres au lancement.
result.execution_id · step_id	UUID	FK execution NOT NULL ; FK step **nullable** (ON DELETE SET NULL).
result.http_status · response_time_ms · success · error	INTEGER · BIGINT · BOOLEAN · VARCHAR(2000)	Résultat d'une requête.
result.step_name · step_method · step_url	VARCHAR(255) · (10) · (2048)	Copie de l'étape (migration 017).
result.captured_at	TIMESTAMPTZ	Horodatage de la requête.

Tables `metric` et `scheduled_execution`

Colonne	Type	Remarque
metric.application_id · scenario_id · step_id · execution_id	UUID	Seul execution_id est NOT NULL ; step_id ON DELETE SET NULL.
metric.response_time · status_code · throughput · error_rate · timestamp	INTEGER · INTEGER · NUMERIC · NUMERIC · TIMESTAMPTZ	Une ligne par résultat.
scheduled_execution.scenario_id · schedule_type · cron_expression · run_at · timezone	UUID · VARCHAR(30) · (120) · TIMESTAMPTZ · VARCHAR(60)	ONE_TIME ou RECURRING_CRON.
scheduled_execution.enabled · next_run_at · last_*	BOOLEAN · TIMESTAMPTZ …	Index sur next_run_at et scenario_id.

Relations et clés étrangères (règles réelles)

Clé étrangère	Référence	ON DELETE	Conséquence
`scenario.application_id`	application	NO ACTION	Cascade faite par le service (une transaction).
`step.scenario_id`	scenario	NO ACTION	Idem.
`execution.scenario_id`	scenario	NO ACTION	Idem.
`scheduled_execution.scenario_id`	scenario	NO ACTION	Idem.
`execution_step_result.execution_id`	execution	NO ACTION	Idem.
`execution_step_result.step_id`	step	**SET NULL**	Supprimer une étape garde l'historique lisible.
`metric.step_id`	step	**SET NULL**	Idem pour les métriques.
`metric.application_id · scenario_id · execution_id`	application · scenario · execution	NO ACTION	Supprimées par la cascade du service.

```
application 1 ──< scenario 1 ──< step
                    │  └──< scheduled_execution
                    └──< execution 1 ──< execution_step_result >── step (SET NULL)
                                   └──< metric >── step (SET NULL)
```

Migrations Liquibase du module

Fichier	Contenu
`003-create-scenario.xml`	Table scenario.
`004-create-step.xml`	Table step.
`005-create-execution.xml`	Table execution.
`006-create-execution-step-result.xml`	Table des résultats.
`007-create-metric.xml`	Table metric.
`010-add-load-test-parameters.xml`	Paramètres de charge (VU, ramp-up, durée, itérations, think time).
`011-create-notification-and-scheduled-execution.xml`	Planifications et `execution.triggered_by`.
`013-add-step-engine-options.xml`	Options par étape et CSV.
`014-add-pacing-and-variable-capture.xml`	Débit cible et capture de variable.
`017-deletion-history-and-archiving.xml`	Copie de l'étape dans les résultats, FK SET NULL, archivage.

> **Constat :** en dehors de `scheduled_execution`, les colonnes de clé étrangère (`step.scenario_id`, `execution.scenario_id`, `execution_step_result.execution_id`, `metric.*`) n'ont **pas d'index** : seules les clés primaires sont indexées.

6. Sécurité

Kol requête khassha JWT dyal Keycloak. Les rôles kaytverifaw f backend b `@PreAuthorize`, w frontend kaykhbbi les boutons li ma 3ndekch l7a9 fihom (ghir confort, backend howa li kay9erer).

Action	SUPER_ADMIN	PERFORMANCE_ENGINEER	VIEWER
Consulter scénarios, étapes, exécutions, rapports, historique, métriques	**Oui**	**Oui**	**Oui**
Créer / modifier un scénario	**Oui**	**Oui**	**Non**
Créer / modifier / supprimer une étape	**Oui**	**Oui**	**Non**
Lancer / annuler / relancer une exécution	**Oui**	**Oui**	**Non**
Voir l'impact, archiver, restaurer un scénario	**Oui**	**Oui**	**Non**
Supprimer définitivement un scénario	**Oui**	**Non**	**Non**
Gérer les planifications (créer, modifier, activer, déclencher, supprimer)	**Oui**	**Oui**	**Non**

> Les identifiants de l'application cible (jeton, clé, mot de passe) ne sont jamais renvoyés par l'API et ne sont pas écrits dans les logs ; le moteur les ajoute uniquement aux requêtes sortantes.

7. Suppression et archivage

Élément	Comportement réel
Étape	Suppression directe (SA, PE). **Refusée** si une exécution du scénario est QUEUED/RUNNING. Les résultats passés gardent leur copie (nom, méthode, URL) et `step_id` passe à NULL. Les étapes restantes sont renumérotées 1..n.
Scénario	**Impact** avant confirmation. **Archiver** (par défaut) : masqué des listes, non exécutable, planifications désactivées, restaurable. **Supprimer définitivement** (SA, saisie du nom) : métriques, résultats, exécutions, planifications, étapes et scénario supprimés dans une seule transaction.
Application	Même logique, cascade sur tous ses scénarios ; un scénario d'une application archivée est masqué et ne peut pas être restauré seul.
Exécution en cours	Toute suppression ou archivage est refusé : « Une exécution est en cours, arrêtez-la avant de supprimer. »
Audit	Actions DELETE, ARCHIVE et RESTORE tracées (succès et échec).

8. Flux complets

A. Créer un scénario (assistant)

```
Informations ─► Étapes ─► Tester ──────────────────────► Charge & résumé
 application      éditeur   POST /api/scenarios (1 VU)        PUT /api/scenarios/{id} (charge choisie)
 nom, desc.       local     POST/PUT /api/steps (ordre 1..n)   « Enregistrer » ou « Enregistrer et lancer »
                            POST /api/executions → suivi → résultats par étape
```

1. Informations : khtar l'application w smiya dyal scénario.
2. Étapes : tzid les étapes (méthode, URL, en-têtes, corps, validation, variables) ; brouillon local.
3. Tester : scénario w étapes kaytsjlo, w kaytlanca test 7a9i9i b 1 VU × 1 passage.
4. Charge & résumé : preset wla paramètres, puis enregistrement (w lancement ila bghiti).

B. Ajouter / modifier des étapes

Men détail dyal scénario (page Scénarios) : modale d'étape → `POST /api/steps` wla `PUT /api/steps/{id}` ; suppression → `DELETE /api/steps/{id}`, renumérotation automatique.

C. Lancer une exécution et suivre

```
POST /api/executions ──► 202 {status: QUEUED|RUNNING}
        │
        ├─ GET /api/executions/{id}/status   (polling : requêtes, succès, échecs, progression)
        ├─ POST /api/executions/{id}/cancel  (annulation réelle)
        ▼
statut final SUCCESS | FAILED | CANCELLED ──► GET /api/executions/{id}/report
```

D. Consulter les résultats

Détail (une ligne par requête), Rapport (statistiques globales, par étape, erreurs, graphiques, export CSV `/report/export`), Historique paginé (`/api/executions/history`).

E. Supprimer / archiver

```
Clic « Supprimer » ─► GET …/deletion-impact ─► modale (compteurs, exécution en cours ?)
     ├─ Archiver ──────────────────► POST …/archive   (restaurable : POST …/restore)
     └─ Supprimer définitivement ─► saisie du nom ─► DELETE …   (SUPER_ADMIN, cascade)
```

9. Exemple concret sur banking-test-api

Exécution **réelle** lancée le 01/10/2026 via l'API LoadPilot, sur une application temporaire (supprimée ensuite) pointant vers `http://localhost:8888`.

Élément	Valeur
Application	URL `http://localhost:8888` · authentification **Connexion auto** : `POST /api/auth/login`, corps `{"username":"agent001","password":"test123"}`, jeton `$.token`. Test de connectivité : **CONNECTED** « Authentification réussie (token obtenu). »
Scénario	« Exemple rapport — Parcours agent » · **5 VU** · ramp-up 5 s · durée 20 s · think time 500 ms

#	Étape	Méthode	URL	En-têtes / validation
1	Consulter le solde	GET	`/api/account/balance`	Auth de l'application ajoutée automatiquement · attendu 200 · contient `"success":true`
2	Historique de l'agent	GET	`/api/agent/history`	Auth automatique · attendu 200
3	Vérifier le jeton	GET	`/api/auth/verify`	En-tête de l'étape `Authorization: Bearer ${authToken}` (prioritaire) · attendu 200 · contient `"authenticated":true`

Résultat obtenu

SUCCESS statut final	**288** requêtes (288 réussies, 0 échec)	**100 %** taux de succès	**13,4 req/s** débit
150 ms temps moyen	**301 ms** P95	**507 ms** P99	**21,5 s** durée réelle

Étape	Requêtes	Réussies	Moyenne	P95	Taux d'erreur
Consulter le solde	96	96	223 ms	308 ms	0 %
Historique de l'agent	96	96	220 ms	305 ms	0 %
Vérifier le jeton	96	96	6 ms	16 ms	0 %

Global : min 2 ms, P50 165 ms, max 610 ms, 0 erreur. Les étapes 1 et 2 sont plus lentes parce que banking-test-api y simule une latence ; `/api/auth/verify` n'en a pas.

10. Tests

605 / 605
`mvn test` backend (0 échec)

319
tests backend du module

51 / 51
`npm test` frontend

OK
`npm run build` (102 modules)

Type	Outil	Tests	Classes du module
API / intégration	Spring Boot Test + MockMvc + H2	134	ScenarioControllerTest (22), StepControllerTest (28), ExecutionControllerTest (23), ExecutionHistoryAndReportTest (18), MetricControllerTest (18), ScheduledExecutionControllerTest (13), DeletionArchivingPagingTest (7), ExecutionConcurrencyLimitsTest (3), ScheduledExecutionCapacityLimitTest (1), EndToEndIntegrationTest (1)
Repositories	@DataJpaTest + H2	44	MetricRepositoryTest (16), ExecutionRepositoryTest (10), ScenarioRepositoryTest (6), StepRepositoryTest (6), ExecutionStepResultRepositoryTest (6)
Moteur (intégration)	JUnit 5 + vrai serveur HTTP local	58	HttpClientExecutionEngineTest (47), ApplicationAuthEngineTest (11)
Unitaires	JUnit 5 + AssertJ	83	PercentileCalculatorTest (14), MetricServiceImplTest (13), JsonPathExtractorTest (11), ScheduleNextRunCalculatorTest (10), CsvDataSourceTest (9), PerformanceStatisticsServiceTest (8), ExecutionTransactionHelperRaceTest (8), VariableResolverTest (7), ExecutionRecoveryRunnerTest (3)
Frontend	Vitest + Testing Library	4	Pagination.test.tsx (4) — composant partagé ; aucun test des pages Scénarios ni de l'assistant.
End-to-end réel	Scripts Python (API réelle + banking-test-api)	manuel	Exemple de la section 9 ; vérification de la suppression/archivage (étape supprimée avec historique lisible, refus en cours, cascade, groupé, pagination).

Ce que vérifient les principales classes

Classe	Vérifie
HttpClientExecutionEngineTest	Méthodes HTTP, en-têtes, URL relatives/absolues, codes attendus, timeouts, échec qui arrête l'itération, 5 et 10 VU réels, ramp-up, durée, itérations, think time, annulation, CSV cyclique, assertions, redirections, débit cible partagé, capture de variable.
ApplicationAuthEngineTest	Login une seule fois pour tous les VU, login refusé, jeton absent, réponse non JSON, `${authToken}`, clé API en paramètre, Basic, priorité de l'en-tête d'étape.
ExecutionHistoryAndReportTest	Historique paginé/filtré, rapport (percentiles, par étape, erreurs) et export CSV avec de vraies exécutions.
ExecutionConcurrencyLimitsTest / CapacityLimitTest	Refus au-delà des limites de capacité, y compris pour les planifications.
ExecutionTransactionHelperRaceTest · RecoveryRunnerTest	Course annulation/fin d'exécution ; exécutions orphelines passées en FAILED au démarrage.
DeletionArchivingPagingTest	Étape supprimée avec historique lisible et renumérotation, refus si exécution en cours, archivage/restauration, cascade, actions groupées.

11. Limites et améliorations possibles

Constat (vérifié dans le code)	Amélioration possible
Statut `INACTIVE` d'un scénario jamais attribué (aucun endpoint pour désactiver) ; le filtre « Inactif » est toujours vide. Le statut d'une étape n'est pas pris en compte par le moteur.	Ajouter activer/désactiver, ou retirer ces statuts.
Page Scénarios : cases à cocher sans action groupée ; « tout sélectionner » prend toute la liste filtrée.	Barre d'actions (archiver, exécuter) comme sur la page Applications.
Liste des scénarios et des étapes non paginée côté serveur (filtrage et pagination dans le navigateur) ; pas de sélecteur de tri.	Pagination serveur comme `/api/applications`.
Aucun test frontend des pages Scénarios, Exécutions et de l'assistant.	Tests Testing Library sur ces écrans.
Colonnes de clé étrangère sans index (sauf `scheduled_execution`).	Migration ajoutant des index sur `scenario_id`, `execution_id`.
Une ligne de résultat **et** une métrique par requête (288 de chaque pour 20 s à 5 VU) : volume qui grossit vite.	Agrégation par fenêtre de temps, rétention ou purge.
Générateur de charge dans la même JVM que l'API (limites 200 VU globaux, 10 exécutions).	Workers d'injection séparés pour les fortes charges.
Connexion auto : un seul login par exécution, pas de renouvellement du jeton.	Rafraîchissement du jeton pour les tests longs.
Capture JSONPath simplifiée (pas de tableaux ni filtres) ; corps de requête texte uniquement (pas de multipart).	Bibliothèque JSONPath complète, support multipart.
Pas de montée/descente progressive en fin de test (ramp-down) ni de profils par paliers.	Profils de charge à étapes.
Restaurer un élément archivé ne réactive pas ses planifications.	Proposer la réactivation à la restauration.
Statistiques du rapport recalculées à chaque consultation depuis tous les résultats.	Mise en cache du rapport d'une exécution terminée.
Bundle frontend de 678 kB (avertissement Vite > 500 kB).	Découpage par route (import dynamique).